

Banco de Dados II (SQL RDBMS)

Recursos Avançados do PostgreSQL:
JSONB, Full-Text Search, RAG (pgvector) e GIS (PostGIS)

Pedro Gabriel Takenobu Serafim
João Victor Torres Soares
Daniel Teles de Oliveira
Rubens Rodrigues Maranesi

PUC-SP – Ciências da Computação
10 de junho de 2026

Atividade Prática em Equipe

Servidor: PostgreSQL Neon (gratuito) | Ambiente: Google Colab
Repositório GitHub: [[inserir link do repositório aqui](#)]

Divisão de tarefas da equipe

Membro	Atividade
Pedro Gabriel Takenobu Serafim	Atividade 1 – JSONB (dados semiestruturados)
João Victor Torres Soares	Atividade 2 – Full-Text Search
Daniel Teles de Oliveira	Atividade 3 – RAG com pgvector
Rubens Rodrigues Maranesi	Atividade 4 – GIS com PostGIS

Sumário

1	Introdução	3
2	Atividade 1 – JSONB: Dados Semiestruturados	3
2.1	Descrição	3
2.2	Scripts SQL principais	3
2.3	Prints de resultados	5
3	Atividade 2 – Full-Text Search	6
3.1	Descrição	6
3.2	Scripts SQL principais	6
3.3	Prints de resultados	7
4	Atividade 3 – RAG com pgvector	9
4.1	Descrição	9
4.2	Scripts SQL principais	9
4.3	Prints de resultados	10
5	Atividade 4 – GIS com PostGIS	15
5.1	Descrição	15
5.2	Scripts SQL principais	15
5.3	Prints de resultados	16
6	Tabela comparativa entre as abordagens	18
7	Dificuldades encontradas e aprendizados	18
8	Conclusão	19

1 Introdução

Este relatório documenta a atividade prática da disciplina de **Banco de Dados II**, cujo objetivo foi explorar recursos avançados do PostgreSQL amplamente utilizados em sistemas corporativos e de grande escala. O trabalho foi desenvolvido em equipe, com cada integrante responsável por uma das quatro frentes tecnológicas.

Todas as atividades foram implementadas **principalmente via SQL**, utilizando um banco de dados criado no serviço gratuito **Neon** e conectado ao **Google Colab** por meio da extensão `%%sql` (*ipython-sql*) ou da biblioteca `psycopg2`. As credenciais foram tratadas de forma segura através do recurso *Colab Secrets* (`google.colab.userdata`).

Seguindo o padrão obrigatório da disciplina, todas as tabelas contêm os campos de controle:

```
1 id          SERIAL PRIMARY KEY,
2 uuid        UUID DEFAULT gen_random_uuid() UNIQUE NOT NULL,
3 created_at  TIMESTAMPTZ DEFAULT NOW() NOT NULL,
4 updated_at  TIMESTAMPTZ DEFAULT NOW() NOT NULL
```

As quatro tecnologias estudadas foram:

- **JSONB** – armazenamento e consulta eficiente de dados semiestruturados;
- **Full-Text Search** – busca textual linguística com ranqueamento e índices;
- **pgvector** + **RAG** – busca semântica por embeddings vetoriais;
- **PostGIS (GIS)** – manipulação de dados geoespaciais e consultas espaciais.

2 Atividade 1 – JSONB: Dados Semiestruturados

Responsável: Pedro Gabriel Takenobu Serafim

2.1 Descrição

O tipo JSONB permite armazenar documentos JSON de forma binária e indexável, combinando a flexibilidade de esquemas dinâmicos com o poder de consulta do SQL. Foi criada a tabela **produtos**, com uma coluna **detalhes** JSONB contendo especificações, avaliações, arrays e objetos aninhados de pelo menos 15 produtos distintos (placas de vídeo, celulares, cafeteiras, drones, smartwatches etc.).

2.2 Scripts SQL principais

Criação da tabela:

```
1 CREATE TABLE produtos (
2     id          SERIAL PRIMARY KEY,
3     uuid        UUID DEFAULT gen_random_uuid() UNIQUE NOT NULL,
4     nome        TEXT NOT NULL,
5     detalhes     JSONB NOT NULL,
6     created_at  TIMESTAMPTZ DEFAULT NOW() NOT NULL,
7     updated_at  TIMESTAMPTZ DEFAULT NOW() NOT NULL
8 );
```

Extração de dados com `->`, `->>` e `#>>`:

```
1 SELECT
2     nome,
3     detalhes -> 'marca'           AS marca_jsonb,
4     detalhes ->> 'marca'          AS marca_texto,
5     detalhes #>> '{especificacoes, ram}' AS ram
```

```
6 FROM produtos
7 ORDER BY nome;
```

Contenção (@>) e existência de chaves (?, ?|, ?&):

```
1 -- Produtos da marca AMD
2 SELECT nome FROM produtos WHERE detalhes @> '{"marca": "AMD"}';
3
4 -- Produtos que possuem simultaneamente avaliacoes e itens_relacionados
5 SELECT nome FROM produtos
6 WHERE detalhes ?& ARRAY['avaliacoes', 'itens_relacionados'];
```

Consultas JSONPath:

```
1 -- Produtos com ao menos uma avaliacao nota 5
2 SELECT nome FROM produtos
3 WHERE jsonb_path_exists(detalhes, '$.avaliacoes[*] ? (@.nota == 5)');
```

Atualização sem reescrever o documento (jsonb_set, jsonb_insert):

```
1 UPDATE produtos
2 SET detalhes = jsonb_set(detalhes, '{preco}', '480.00'),
3     updated_at = NOW()
4 WHERE nome = 'Drone';
```

Agregação com jsonb_array_elements (média de notas por produto):

```
1 SELECT p.nome,
2        COUNT(*) AS total_avaliacoes,
3        ROUND(AVG((av.value ->> 'nota')::NUMERIC), 2) AS media_nota
4 FROM produtos p,
5        jsonb_array_elements(p.detalhes -> 'avaliacoes') AS av
6 GROUP BY p.nome
7 ORDER BY media_nota DESC;
```

Índice GIN e análise de desempenho:

```
1 CREATE INDEX idx_detalhes_gin ON produtos USING GIN (detalhes);
2
3 EXPLAIN ANALYZE
4 SELECT nome FROM produtos WHERE detalhes @> '{"marca": "AMD"}';
```

2.3 Prints de resultados

Consulta 1 — Extração com `->`, `->>` e `#>>`

O operador `->` retorna o valor como JSONB (preserva o tipo original). O `->>` retorna como TEXT, útil para comparações e exibição. O `#>>` navega por um caminho aninhado definido como array de chaves e retorna TEXT. Aqui extraímos a marca em dois formatos e o campo ram que está dois níveis abaixo, dentro de especificacoes.

```
%%sql
SELECT
  nome,
  detalhes -> 'marca'           AS marca_jsonb,
  detalhes ->> 'marca'         AS marca_texto,
  detalhes ->> 'preco'          AS preco,
  detalhes #>> '{especificacoes, ram}' AS ram
FROM produtos
ORDER BY nome;
```

* postgresql://neondb_owner:***@ep-raspy-star-apmmeehl-pooler.c-7.us-east-1.aws.neon.tech/neondb?channel_binding=require&sslmode=require
15 rows affected.

nome	marca_jsonb	marca_texto	preco	ram
Cafeteira	CafePerfeito	CafePerfeito	349.00	None
Caixa de Som Bluetooth	SoundWave	SoundWave	69.99	None
Carregador Wireless	PowerUp	PowerUp	39.90	None
Celular	Apple	Apple	7599.99	None
Câmera Mirrorless	FotoMax	FotoMax	8500.00	None
Drone	FlyHigh	FlyHigh	450.00	None
E-reader	ReadSmart	ReadSmart	189.99	None
HD Externo	DataStore	DataStore	119.00	None
Hub Smart Home	CasaConectada	CasaConectada	89.90	None
Notebook	WorkLap	WorkLap	3299.00	16GB DDR5
Placa de Vídeo	AMD	AMD	1299.99	8GB
Smart Band	MoveWell	MoveWell	79.99	None
Smartwatch	HealthTech	HealthTech	299.50	None
Tablet	TabPro	TabPro	399.00	6GB
Videogame	PlayFun	PlayFun	599.99	None

Figura 1: Extração com `->`, `->>` e `#>>`

Consulta 15 — `jsonb_array_elements`: média de notas por produto

`jsonb_array_elements` expande cada elemento de um array JSONB em uma linha separada, funcionando como um CROSS JOIN. Aqui expandimos o array `avaliacoes` de cada produto, convertemos nota para NUMERIC e calculamos a média agrupando por produto.

```
%%sql
SELECT
  p.nome,
  COUNT(*) AS total_avaliacoes,
  ROUND(AVG((av.value ->> 'nota')::NUMERIC), 2) AS media_nota
FROM produtos p,
  jsonb_array_elements(p.detalhes -> 'avaliacoes') AS av
GROUP BY p.nome
ORDER BY media_nota DESC;
```

... * postgresql://neondb_owner:***@ep-odd-fog-ap6345ba-pooler.c-7.us-east-1.aws.neon.tech/neondb?channel_binding=require&sslmode=require
15 rows affected.

nome	total_avaliacoes	media_nota
E-reader	2	5.00
Celular	3	4.67
Smartwatch	2	4.50
Placa de Vídeo	2	4.50
Drone	2	4.50
Cafeteira	2	4.50
Câmera Mirrorless	2	4.50
Notebook	2	4.50
Smart Band	2	4.50
Caixa de Som Bluetooth	2	4.50
HD Externo	2	4.50
Videogame	2	4.50
Tablet	3	4.00
Hub Smart Home	2	3.50
Carregador Wireless	2	3.50

Figura 2: Média de notas por produto (`jsonb_array_elements`)

```
Consulta 19 — EXPLAIN ANALYZE com @>

EXPLAIN ANALYZE executa a consulta e exibe o plano de execução com tempos reais. Após criar o índice GIN, esperamos ver Bitmap
Index Scan no plano, indicando que o índice foi usado. Sem o índice, apareceria Seq Scan, que percorre todos os registros um a um.

%sql
EXPLAIN ANALYZE
SELECT nome
FROM produtos
WHERE detalhes @> '{"marca": "AMD"}';

... * postgresql://neondb_owner:***@ep-odd-fog-ap6345ba-pooler.c-7.us-east-1.aws.neon.tech/neondb?channel_binding=require&sslmode=require
8 rows affected.

QUERY PLAN
Seq Scan on produtos (cost=0.00..2.19 rows=1 width=32) (actual time=0.011..0.014 rows=1 loops=1)
  Filter: (detalhes @> '{"marca": "AMD"}'::jsonb)
  Rows Removed by Filter: 14
  Buffers: shared hit=2
Planning:
  Buffers: shared hit=24
Planning Time: 0.114 ms
Execution Time: 0.029 ms
```

Figura 3: EXPLAIN ANALYZE mostrando o uso do índice GIN (Bitmap Index Scan)

3 Atividade 2 – Full-Text Search

Responsável: João Victor Torres Soares

3.1 Descrição

O **Full-Text Search (FTS)** é o mecanismo nativo de busca textual do PostgreSQL. Diferentemente de LIKE/ILIKE, ele processa linguisticamente o texto – remove *stop words*, aplica *stemming* e registra a posição das palavras – armazenando o resultado em uma coluna **TSVECTOR**. Como base de dados foram utilizados três livros do Project Gutenberg (*Alice no País das Maravilhas*, *Dom Casmurro* e *Quincas Borba*), divididos em parágrafos. Cada parágrafo virou um registro independente, com a configuração de idioma adequada (*english/portuguese*).

3.2 Scripts SQL principais

Criação da tabela e geração do TSVECTOR:

```
1 CREATE TABLE documentos_texto (
2     id SERIAL PRIMARY KEY,
3     uuid UUID DEFAULT gen_random_uuid() UNIQUE NOT NULL,
4     titulo TEXT NOT NULL,
5     conteudo TEXT NOT NULL,
6     ts_conteudo TSVECTOR,
7     created_at TIMESTAMPTZ DEFAULT NOW() NOT NULL,
8     updated_at TIMESTAMPTZ DEFAULT NOW() NOT NULL
9 );
10
11 UPDATE documentos_texto
12 SET ts_conteudo = CASE
13     WHEN idioma = 'english' THEN to_tsvector('english', conteudo)
14     WHEN idioma = 'portuguese' THEN to_tsvector('portuguese', conteudo)
15 END;
```

Índice GIN:

```
1 CREATE INDEX idx_ts_conteudo ON documentos_texto USING GIN(ts_conteudo);
```

Busca com operadores lógicos (AND &, OR |, NOT !) e prefixo (:*):

```
1 -- AND: documentos com rabbit E queen
2 SELECT titulo, LEFT(conteudo,1000) AS trecho
```

```

3 FROM documentos_texto
4 WHERE ts_conteudo @@ to_tsquery('english', 'rabbit & queen');
5
6 -- Prefixo: rabbit, rabbits, ...
7 SELECT LEFT(conteudo,200) AS trecho
8 FROM documentos_texto
9 WHERE ts_conteudo @@ to_tsquery('english', 'rab:*');

```

Ranqueamento (ts_rank, ts_rank_cd) e destaque (ts_headline):

```

1 SELECT LEFT(conteudo,500) AS trecho,
2        ts_rank(ts_conteudo, to_tsquery('english','rabbit')) AS relevancia
3 FROM documentos_texto
4 WHERE ts_conteudo @@ to_tsquery('english','rabbit')
5 ORDER BY relevancia DESC LIMIT 10;
6
7 SELECT ts_headline('english', conteudo, to_tsquery('english','cat')) AS
8        destaque
9 FROM documentos_texto
10 WHERE ts_conteudo @@ to_tsquery('english','cat') LIMIT 5;

```

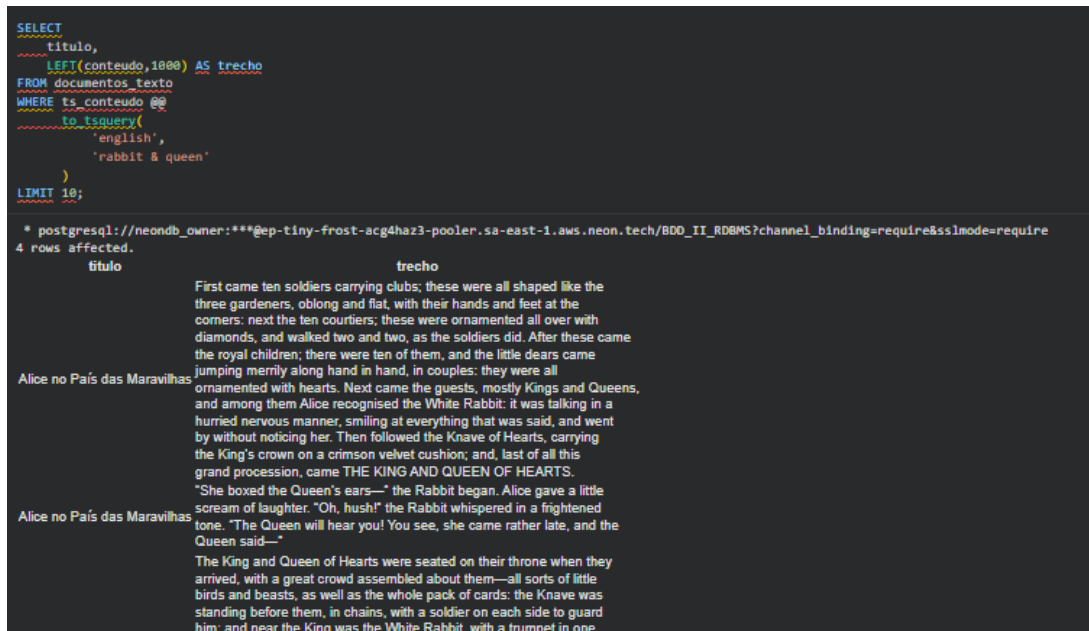
Comparação de desempenho FTS vs LIKE/ILIKE:

```

1 EXPLAIN ANALYZE SELECT * FROM documentos_texto WHERE conteudo ILIKE '%rabbit%';
2
3 EXPLAIN ANALYZE SELECT * FROM documentos_texto
4 WHERE ts_conteudo @@ to_tsquery('english', 'rabbit');

```

3.3 Prints de resultados



```

SELECT
  titulo,
  LEFT(conteudo,1000) AS trecho
FROM documentos_texto
WHERE ts_conteudo @@
      to_tsquery(
        'english',
        'rabbit & queen'
      )
LIMIT 10;

```

* postgresql://neondb_owner:***@ep-tiny-frost-acg4haz3-pooler.sa-east-1.aws.neon.tech/BDO_TI_RDBMS?channel_binding=require&sslmode=require
4 rows affected.

titulo	trecho
Alice no País das Maravilhas	First came ten soldiers carrying clubs; these were all shaped like the three gardeners, oblong and flat, with their hands and feet at the corners: next the ten courtiers; these were ornamented all over with diamonds, and walked two and two, as the soldiers did. After these came the royal children; there were ten of them, and the little dears came jumping merrily along hand in hand, in couples: they were all ornamented with hearts. Next came the guests, mostly Kings and Queens, and among them Alice recognised the White Rabbit: it was talking in a hurried nervous manner, smiling at everything that was said, and went by without noticing her. Then followed the Knave of Hearts, carrying the King's crown on a crimson velvet cushion; and, last of all this grand procession, came THE KING AND QUEEN OF HEARTS.
Alice no País das Maravilhas	"She boxed the Queen's ears—" the Rabbit began. Alice gave a little scream of laughter. "Oh, hush!" the Rabbit whispered in a frightened tone. "The Queen will hear you! You see, she came rather late, and the Queen said—"
	The King and Queen of Hearts were seated on their throne when they arrived, with a great crowd assembled about them—all sorts of little birds and beasts, as well as the whole pack of cards: the Knave was standing before them, in chains, with a soldier on each side to guard him; and near the King was the White Rabbit, with a trumpet in one

Figura 4: Busca com operador AND (rabbit & queen)

```

SELECT
  LEFT(conteudo,10000) AS trecho,
  ts_rank(
    ts_conteudo,
    to_tsquery('english','rabbit')
  ) AS relevancia
FROM documentos_texto
WHERE ts_conteudo @@
  to_tsquery('english','rabbit')
ORDER BY relevancia DESC
LIMIT 10;

```

* postgresql://neondb_owner:***@ep-tiny-frost-acg4haz3-pooler.sa-east-1.aws.neon.tech/BDD_II_RDBMS?channel_binding=require&sslmode=require
10 rows affected.

trecho	relevancia
There was nothing so .very .remarkable in that; nor did Alice think it so .very .much out of the way to hear the Rabbit say to itself, "Oh dear! Oh dear! I shall be late!" (when she thought it over afterwards, it occurred to her that she ought to have wondered at this, but at the time it all seemed quite natural); but when the Rabbit actually .took a watch out of its waistcoat-pocket , and looked at it, and then hurried on, Alice started to her feet, for it flashed across her mind that she had never before seen a rabbit with either a waistcoat-pocket, or a watch to take out of it, and burning with curiosity, she ran across the field after it, and fortunately was just in time to see it pop down a large rabbit-hole under the hedge.	0.08054518
There was nothing so .very .remarkable in that; nor did Alice think it so .very .much out of the way to hear the Rabbit say to itself, "Oh dear! Oh dear! I shall be late!" (when she thought it over afterwards, it occurred to her that she ought to have wondered at this, but at the time it all seemed quite natural); but when the Rabbit actually .took a watch out of its waistcoat-pocket , and looked at it, and then hurried on, Alice started to her feet, for it flashed across her mind that she had never before seen a rabbit with either a waistcoat-pocket, or a watch to take out of it, and burning with curiosity, she ran across the field after it, and fortunately was just in time to see it pop down a large rabbit-hole under the hedge.	0.08054518
After a time she heard a little pattering of feet in the distance, and she hastily dried her eyes to see what was coming. It was the White Rabbit, and he was looking at his watch.	

Figura 5: Ranqueamento por relevância com *ts_rank*

```

Xxsql

SELECT
  ts_headline(
    'english',
    conteudo,
    to_tsquery('english','cat')
  ) AS destaque
FROM documentos_texto
WHERE ts_conteudo @@
  to_tsquery('english','cat')
LIMIT 5;

```

* postgresql://neondb_owner:***@ep-tiny-frost-acg4haz3-pooler.sa-east-1.aws.neon.tech/BDD_II_RDBMS?channel_binding=require&sslmode=require
5 rows affected.

destaque
cats eat bats? Do cats eat bats?" and sometimes, "Do bats eat cats?" for, you see, as she couldn
cats eat bats? Do cats eat bats?" and sometimes, "Do bats eat cats?" for, you see, as she couldn
always
hated cats; nasty, low, vulgar things! Don't let me hear the name again!"
that she
had hurt the poor animal's feelings. "I quite forgot you didn't like cats."
cats!" cried the Mouse, in a shrill, passionate voice. "Would
you like cats if you were

Figura 6: Destaque de termos com *ts_hheadline*


```
XXsql
EXPLAIN ANALYZE

SELECT *
FROM documentos_texto
WHERE conteudo ILIKE '%rabbit%';

* postgresql://neondb_owner:***@ep-tiny-frost-acg4haz3-pooler.sa-east-1.aws.neon.tech/BDD_II_RDBMS?channel_binding=require&sslmode=require
8 rows affected.

QUERY PLAN
Seq Scan on documentos_texto (cost=0.00..405.68 rows=102 width=539) (actual time=0.012..2.522 rows=42.00 loops=1)
  Filter: (conteudo ~* '%rabbit%':text)
  Rows Removed by Filter: 1852
  Buffers: shared hit=388
Planning:
  Buffers: shared hit=15 dirtied=2
Planning Time: 0.323 ms
Execution Time: 2.544 ms

XXsql
EXPLAIN ANALYZE

SELECT *
FROM documentos_texto
WHERE conteudo LIKE '%rabbit%';

* postgresql://neondb_owner:***@ep-tiny-frost-acg4haz3-pooler.sa-east-1.aws.neon.tech/BDD_II_RDBMS?channel_binding=require&sslmode=require
6 rows affected.

QUERY PLAN
Seq Scan on documentos_texto (cost=0.00..405.68 rows=23 width=539) (actual time=0.021..1.461 rows=6.00 loops=1)
  Filter: (conteudo ~ '%rabbit%':text)
  Rows Removed by Filter: 1888
  Buffers: shared hit=388
Planning Time: 0.196 ms
Execution Time: 1.476 ms
```

Figura 7: EXPLAIN ANALYZE: FTS (índice GIN) vs ILIKE (Seq Scan)

4 Atividade 3 – RAG com pgvector

Responsável: Daniel Teles de Oliveira

4.1 Descrição

O **RAG (Retrieval-Augmented Generation)** combina recuperação de informação com geração de texto por LLMs. A recuperação foi implementada via **embeddings vetoriais** usando a extensão **pgvector**. Os mesmos três livros foram divididos em *chunks* de aproximadamente 500 caracteres; cada chunk foi convertido em um vetor de 384 dimensões pelo modelo **all-MiniLM-L6-v2** (*sentence-transformers*) e armazenado junto a metadados em JSONB.

4.2 Scripts SQL principais

Extensão, tabela e índice HNSW:

```
1 CREATE EXTENSION IF NOT EXISTS vector;
2
3 CREATE TABLE documentos_vetor (
4     id          SERIAL PRIMARY KEY,
5     uuid        UUID DEFAULT gen_random_uuid() UNIQUE NOT NULL,
6     conteudo     TEXT NOT NULL,
7     embedding    VECTOR(384),
8     metadata     JSONB,
9     created_at   TIMESTAMPTZ DEFAULT NOW() NOT NULL,
10    updated_at   TIMESTAMPTZ DEFAULT NOW() NOT NULL
11 );
12
13 CREATE INDEX idx_embedding ON documentos_vetor
14 USING hnsw (embedding vector_cosine_ops);
```

Busca por similaridade (distância de cosseno <=>):

```
1 SELECT conteudo, metadata FROM documentos_vetor
2 ORDER BY embedding <=> '[...vetor da consulta...]' LIMIT 5;
```

Diferentes métricas – cosseno (<=>), L2 (<->), inner product (<#>):

```
1 ORDER BY embedding <=> :v -- distância de cosseno
2 ORDER BY embedding <-> :v -- distância euclidiana (L2)
3 ORDER BY embedding <#> :v -- produto interno
```

Filtros combinados (vetor + JSONB + texto):

```
1 -- Vetor + filtro JSONB por livro
2 SELECT conteudo, metadata FROM documentos_vetor
3 WHERE metadata->>'livro' = 'Dom Casmurro'
4 ORDER BY embedding <=> :v LIMIT 2;
5
6 -- Vetor + filtro textual
7 SELECT conteudo FROM documentos_vetor
8 WHERE conteudo LIKE '%Bentinho%'
9 ORDER BY embedding <=> :v LIMIT 5;
```

4.3 Prints de resultados

```
def busca_similaridade(query):
    conn = psycopg2.connect(DBURL)
    cursor = conn.cursor()
    vetor_query = modelo.encode(query).tolist()
    vetor_string = str(vetor_query)
    cursor.execute("SELECT conteudo, metadata FROM documentos_vetor ORDER BY embedding <=> %s LIMIT 5;", (vetor_string,))
    resultados = cursor.fetchall()
    for conteudo, metadata in resultados:
        print(f"\nConteúdo: \n\n {conteudo[0:200]}")
        print(f"\nMetadata: \n\n {metadata}")

    cursor.close()
    conn.close()
```

Figura 8: Código da busca semântica retornando os 5 chunks mais próximos)

```
busca_similaridade(query_alice)
```

Conteúdo:

And she went on planning to herself how she would manage it. "They must go by the carrier," she thought; "and how funny it'll seem, sending presents to one's own feet! And how odd the directions will

Metadata:

```
{'chunk': 24, 'livro': "Alice's Adventures in Wonderland"}
```

Conteúdo:

There were doors all round the hall, but they were all locked; and when Alice had been all the way down one side and up the other, trying every door, she walked sadly down the middle, wondering how sh

Metadata:

```
{'chunk': 11, 'livro': "Alice's Adventures in Wonderland"}
```

Conteúdo:

"Mary Ann! Mary Ann!" said the voice. "Fetch me my gloves this moment!" Then came a little pattering of feet on the stairs. Alice knew it was the Rabbit coming to look for her, and she trembled till s

Metadata:

```
{'chunk': 77, 'livro': "Alice's Adventures in Wonderland"}
```

Figura 9: Código da busca semântica retornando os 5 chunks mais próximos

```
query_metricas = "O que aconteceu com Quincas Borba?"  
vetor_query_metricas = modelo.encode(query_metricas).tolist()  
vetor_string = str(vetor_query_metricas)
```

Figura 10: Query para medir as métricas entre Coisine, L2 e Inner Products

```
conn = psycopg2.connect(DBURL)
cursor = conn.cursor()
cursor.execute("SELECT conteudo, metadata, embedding <=> %s as distancia FROM documentos_vetor ORDER BY distancia LIMIT 1;", (vetor_string,))
resultados = cursor.fetchall()
for conteudo, metadata, embedding in resultados:
    print(f"\nConteúdo: \n\n {conteudo}" + f"\nMetadata: \n\n {metadata}" + f"\n\nDistancia: \n\n {embedding}")
cursor.close()
conn.close()
```

Conteúdo:

Ultimamente não me falavam já do seminário, a tal ponto que eu suppunha ser negocio findo. Quinze annos, não havendo vocação, pediam antes o seminário do mundo que o de S. José. Minha mãe ficava muita vez a olhar para mim, como alma perdida, ou pegava-me na mão, a pretexto de nada, para apertal-a muito.

XII

Na varanda.

Metadata:

{'chunk': 58, 'livro': 'Dom Casmurro'}

Distancia:

0.37592875957489014

Figura 11: Retorno da query com Coisine e sua distância

```
conn = psycopg2.connect(DBURL)
cursor = conn.cursor()
cursor.execute("EXPLAIN ANALYZE SELECT conteudo, metadata FROM documentos_vetor ORDER BY embedding <=> %s LIMIT 1;", (vetor_string,))
resultados = cursor.fetchall()
for linha in resultados:
    print(linha[0])
cursor.close()
conn.close()
```

Limit (cost=333.23..333.23 rows=1 width=537) (actual time=4.958..4.959 rows=1.00 loops=1)
Buffers: shared hit=4646
-> Sort (cost=333.23..338.40 rows=2070 width=537) (actual time=4.956..4.957 rows=1.00 loops=1)
Sort Key: ((embedding <=> '[0.014365896,0.053145744,-0.04407887,-0.050969895,-0.08428955,-0.0069729835,0.03662392,0.099607766,0.012772343,0.022...
Sort Method: top-N heapsort Memory: 25kB
Buffers: shared hit=4646
-> Seq Scan on documentos_vetor (cost=0.00..322.88 rows=2070 width=537) (actual time=0.040..4.318 rows=2070.00 loops=1)
Buffers: shared hit=4643

Planning:
Buffers: shared hit=162
Planning Time: 0.451 ms
Execution Time: 4.998 ms

Figura 12: Métricas de Coisine

```
conn = psycopg2.connect(DBURL)
cursor = conn.cursor()
cursor.execute("SELECT conteudo, metadata, embedding <-> %s as distancia FROM documentos_vetor ORDER BY distancia LIMIT 1;", (vetor_string,))
resultados = cursor.fetchall()
for conteudo, metadata, embedding in resultados:
    print(f"\nConteúdo: \n\n {conteudo}" + f"\nMetadata: \n\n {metadata}" + f"\n\nDistancia: \n\n {embedding}")
cursor.close()
conn.close()
```

Conteúdo:

Ultimamente não me falavam já do seminário, a tal ponto que eu suppunha ser negocio findo. Quinze annos, não havendo vocação, pediam antes o seminário do mundo que o de S. José. Minha mãe ficava muita vez a olhar para mim, como alma perdida, ou pegava-me na mão, a pretexto de nada, para apertal-a muito.

XII

Na varanda.

Metadata:

{'chunk': 58, 'livro': 'Dom Casmurro'}

Distancia:

0.8670971797611732

Figura 13: Retorno da query com L2 e sua distância


```
busca_similaridade(query_alice)
```

Conteúdo:

And she went on planning to herself how she would manage it. "They must go by the carrier," she thought; "and how funny it'll seem, sending presents to one's own feet! And how odd the directions will

Metadata:

```
{'chunk': 24, 'livro': "Alice's Adventures in Wonderland"}
```

Conteúdo:

There were doors all round the hall, but they were all locked; and when Alice had been all the way down one side and up the other, trying every door, she walked sadly down the middle, wondering how sh

Metadata:

```
{'chunk': 11, 'livro': "Alice's Adventures in Wonderland"}
```

Conteúdo:

"Mary Ann! Mary Ann!" said the voice. "Fetch me my gloves this moment!" Then came a little pattering of feet on the stairs. Alice knew it was the Rabbit coming to look for her, and she trembled till s

Metadata:

```
{'chunk': 77, 'livro': "Alice's Adventures in Wonderland"}
```

Figura 17: Filtro combinado vetor + JSONB (livro = Dom Casmurro)

```
def simulacao_RAG(contexto):
    conn = psycopg2.connect(DBURL)
    cursor = conn.cursor()
    vetor_consulta = modelo.encode(contexto).tolist()
    vetor_string = str(vetor_consulta)
    cursor.execute("SELECT conteudo FROM documentos_vetor ORDER BY embedding <=> %s LIMIT 5;", (vetor_string,))
    resultados = cursor.fetchall()
    for r in resultados:
        print(r[0][:900])
        print("-" * 50)

    cursor.close()
    conn.close()

simulacao_RAG("Quem foi Capitô?")

CAPITULO CII
-----
--Capitô!

--Mamãe!

--Deixa de estar esburacando o muro; vem cá.
-----
Tudo o que contei no fim do outro capítulo foi obra de um instante.
O que se lhe seguiu foi ainda mais rápido. Dei um pulo, e antes que
ella raspasse o muro, li estes dous nomes, abertos ao prego, o assim
dispostos:

BENTO CAPITOLINA
-----
Não podemos deixar de rir, eu mais que ninguém. A primeira pessoa que
fechou a cara, que o repreendeu e chamou a si foi Capitô.

--Não quero isso, ouviu?
```

Figura 18: Simulação de RAG recuperando o contexto da pergunta

5 Atividade 4 – GIS com PostGIS

Responsável: Rubens Rodrigues Maranesi

5.1 Descrição

A extensão **PostGIS** adiciona suporte a dados geoespaciais ao PostgreSQL. Foi criada a tabela **localizacoes** com uma coluna **geom** **GEOMETRY(GEOMETRY, 4326)** (SRID 4326 = WGS84), capaz de armazenar **Points**, **LineStrings** (rotas) e **Polygons** (áreas). Foram inseridos 17 registros com foco no Sudeste do Brasil: 10 pontos (capitais, cidades do interior, pontos turísticos e a PUC-SP), 3 rotas rodoviárias e 4 polígonos (zonas de entrega e regiões).

5.2 Scripts SQL principais

Extensão, tabela e índice GIST:

```
1 CREATE EXTENSION IF NOT EXISTS postgis;
2
3 CREATE TABLE localizacoes (
4     id          SERIAL PRIMARY KEY,
5     uuid        UUID DEFAULT gen_random_uuid() UNIQUE NOT NULL,
6     nome        TEXT NOT NULL,
7     descricao   TEXT,
8     tipo        TEXT,                -- 'ponto', 'rota' ou 'area'
9     geom        GEOMETRY(GEOMETRY, 4326),
10    created_at   TIMESTAMPTZ DEFAULT NOW() NOT NULL,
11    updated_at   TIMESTAMPTZ DEFAULT NOW() NOT NULL
12 );
13
14 CREATE INDEX idx_localizacoes_geom ON localizacoes USING GIST (geom);
```

Inserção de Point, LineString e Polygon:

```

1 -- Ponto
2 INSERT INTO localizacoes (nome, tipo, geom) VALUES
3 ('São Paulo', 'ponto', ST_SetSRID(ST_MakePoint(-46.6333, -23.5505), 4326));
4
5 -- Rota (LineString)
6 INSERT INTO localizacoes (nome, tipo, geom) VALUES
7 ('Rota SP -> RJ', 'rota',
8  ST_SetSRID(ST_MakeLine(ARRAY[
9      ST_MakePoint(-46.6333, -23.5505),
10     ST_MakePoint(-43.1729, -22.9068)]), 4326));
11
12 -- Área (Polygon)
13 INSERT INTO localizacoes (nome, tipo, geom) VALUES
14 ('Zona Central SP', 'area',
15  ST_GeomFromText('POLYGON((-46.66 -23.56, -46.61 -23.56,
16    -46.61 -23.52, -46.66 -23.52, -46.66 -23.56))', 4326));

```

Consultas espaciais (distância, raio, interseção, comprimento):

```

1 -- Distância SP -> RJ em km (cast para geography)
2 SELECT ROUND((ST_Distance(a.geom::geography, b.geom::geography)/1000)::
3     numeric,1)
4 FROM localizacoes a, localizacoes b
5 WHERE a.nome='São Paulo' AND b.nome='Rio de Janeiro';
6
7 -- Pontos a até 120 km de São Paulo
8 SELECT b.nome FROM localizacoes a JOIN localizacoes b
9     ON a.nome='São Paulo' AND b.tipo='ponto'
10    AND ST_DWithin(a.geom::geography, b.geom::geography, 120000);
11
12 -- Rotas que cruzam uma área
13 SELECT r.nome FROM localizacoes r JOIN localizacoes area
14     ON area.nome='Triângulo SP-RJ-BH'
15    WHERE r.tipo='rota' AND ST_Intersects(r.geom, area.geom);
16
17 -- Comprimento das rotas
18 SELECT nome, ROUND((ST_Length(geom::geography)/1000)::numeric,1) AS km
19 FROM localizacoes WHERE tipo='rota';

```

Análise de desempenho:

```

1 EXPLAIN ANALYZE
2 SELECT b.nome FROM localizacoes a, localizacoes b
3 WHERE a.nome='São Paulo' AND b.tipo='ponto'
4     AND ST_DWithin(a.geom::geography, b.geom::geography, 120000);

```

5.3 Prints de resultados

Consulta 12 – Ponto médio de uma rota (ST_LineInterpolatePoint)

Calcula o ponto a 50% do trajeto da Rota SP → RJ.

```

%sql
SELECT nome,
       ST_AsText(ST_LineInterpolatePoint(geom, 0.5)) AS ponto_medio
FROM localizacoes
WHERE nome = 'Rota SP → RJ';

```

* postgresql://neondb_owner:***@ep-steep-breeze-apm3f9n8-pooler.c-7.us-east-1.aws.neon.tech/neondb?channel_binding=require&sslmode=require
1 rows affected.

nome	ponto_medio
Rota SP → RJ	POINT(-44.89174682225054 -22.805048093350326)

Figura 19: Distância SP -> RJ e pontos dentro de raio (ST_DWithin)



Figura 20: Comprimento das rotas (ST_Length)

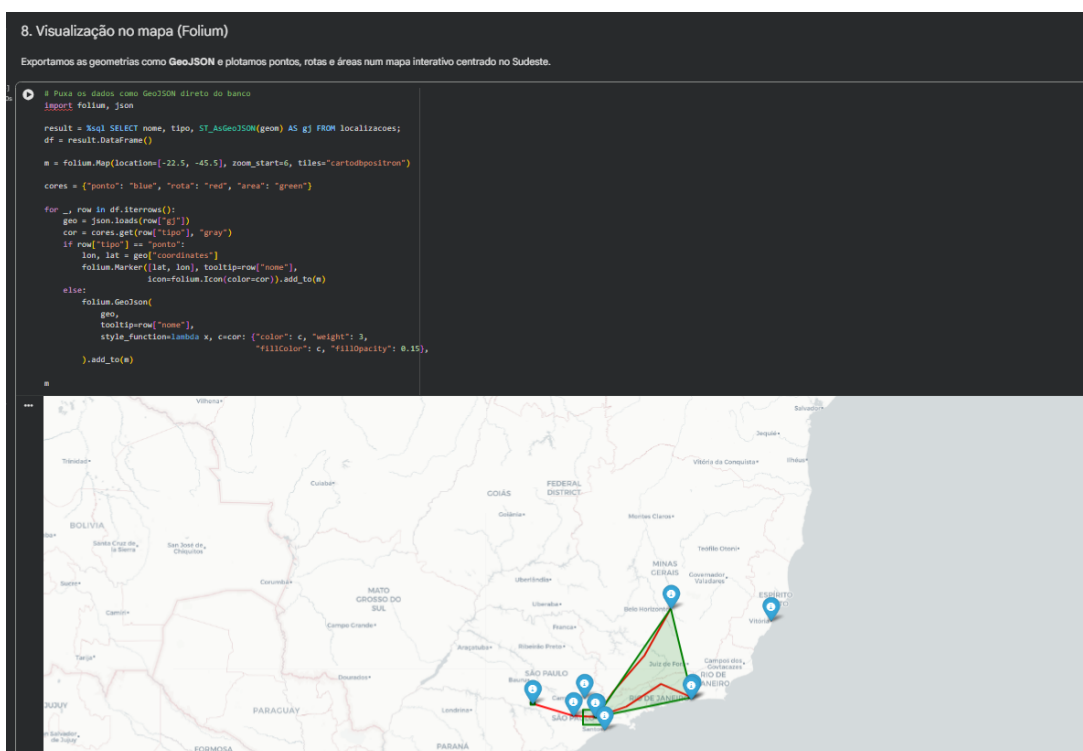


Figura 21: Visualização no mapa interativo (Folium / GeoJSON)



Figura 22: EXPLAIN ANALYZE da consulta espacial com índice GIST

6 Tabela comparativa entre as abordagens

Tabela 1: Comparativo entre JSONB, Full-Text Search, RAG (pgvector) e GIS (PostGIS)

Critério	JSONB	Full-Text Search	RAG (pgvector)	GIS (PostGIS)
Tipo de dado	Documentos JSON	Texto / TSVECTOR	Vetores (embeddings)	Geometrias espaciais
Índice ideal	GIN	GIN	HNSW / IVFFlat	GIST
Busca	Estrutural / por campo	Linguística por palavra	Semântica por significado	Espacial / geográfica
Operadores chave	-> @? ?	@@ & !	<=> <-> <#>	ST_*
Caso de uso	Catálogos, configs flexíveis	Busca em documentos	IA, chatbots, RAG	Mapas, logística, rotas
Vantagem	Esquema flexível	Relevância e <i>stemming</i>	Entende sinônimos/contexto	Cálculos geográficos reais
Limitação	Sem validação rígida	Não capta semântica	Custo de gerar embeddings	Curva de aprendizado

7 Dificuldades encontradas e aprendizados

- **Configuração e credenciais:** conectar o Colab ao Neon exigiu atenção à *connection string* (parâmetros `sslmode=require` e `channel_binding=require`). O uso do *Colab Secrets* foi essencial para não expor senhas no notebook.
- **JSONB:** entender a diferença entre `->` (retorna JSONB) e `->»` (retorna TEXT) foi um ponto de atenção, principalmente ao comparar valores numéricos, que exigem *cast* explícito (`::NUMERIC`).
- **Full-Text Search:** a escolha correta da configuração de idioma (`portuguese` vs `english`)

afeta diretamente o *stemming* e a remoção de *stop words*; misturar idiomas no mesmo `to_tsvector` degradava os resultados.

- **RAG:** a etapa de *chunking* foi decisiva – chunks muito pequenos perdiam contexto e chunks grandes diluíam a relevância. Também foi necessário garantir que a dimensão do vetor (384) batesse com a do modelo de embeddings.
- **GIS:** a ordem das coordenadas em `ST_MakePoint(longitude, latitude)` é uma fonte comum de erro. Aprendemos a sempre fixar o SRID com `ST_SetSRID` e a fazer *cast* para `::geography` quando o objetivo era medir distâncias reais em metros.
- **Índices:** em tabelas pequenas o otimizador às vezes preferiu *Seq Scan*; o `EXPLAIN ANALYZE` foi fundamental para entender quando cada índice (GIN, HNSW, GIST) realmente entra em ação.
- **Trabalho em equipe:** dividir uma atividade por integrante, mas compartilhar o mesmo banco Neon, exigiu coordenação de nomes de tabelas e padronização dos campos de controle obrigatórios.

8 Conclusão

A atividade permitiu explorar, de forma prática, quatro recursos avançados do PostgreSQL que vão muito além do modelo relacional tradicional. Ficou evidente que um único SGBD é capaz de atender a necessidades bastante distintas: armazenamento flexível com **JSONB**, busca textual inteligente com **Full-Text Search**, busca semântica para aplicações de IA com **pgvector/RAG** e análise geoespacial com **PostGIS**.

Cada abordagem mostrou-se mais adequada a um cenário específico, e o uso correto de **índices** (GIN, HNSW e GIST) foi um fator determinante de desempenho em todas elas, comprovado pelos planos de execução obtidos com `EXPLAIN ANALYZE`. O resultado reforça por que essas competências têm alta demanda no mercado, presentes em e-commerce, plataformas de busca, sistemas de recomendação, aplicações de IA generativa e soluções de logística e *smart cities*.